Spotting Faulty Wafers Early: Our Approach

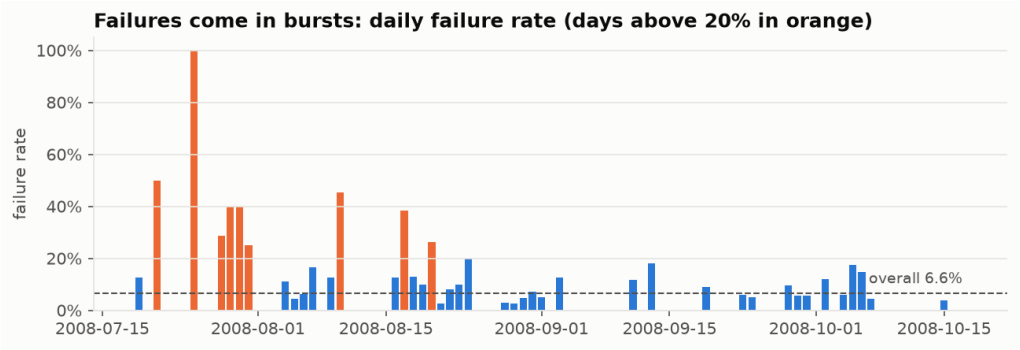
Team Trust Me Bro · DATA DIVERGENCE by DEVS REC · Track 3 (Advanced): SECOM wafer quality

In short

A chip factory makes thousands of wafers. Each wafer passes through hundreds of machines, and 590 sensors record what happened along the way. A few wafers fail the final quality test. Our model reads the sensor readings, plus *when* the wafer was made, and estimates how likely the wafer is to fail. In our tests it **catches about 3 out of 4 faulty wafers while sending about 1 in 3 wafers for an extra check**. It also beats the published benchmark for this dataset.

1. What we found in the data

- **Failures are rare.** Only 83 of 1,253 wafers (6.6%) failed. A lazy model that calls every wafer "good" is right 93% of the time but catches nothing, so we judged models on how well they find the *bad* wafers.
- **No single sensor gives the answer.** The best single sensor only separates good from bad wafers a little (AUC 0.70, where 0.5 is a coin flip). The warning signs are small and spread across many sensors. Some sensors are also often empty, and 116 never change.
- **Failures come in bursts.** If a wafer made within 2 hours of yours failed, your wafer fails 14% of the time, against 5% otherwise: **2.8 times more likely**. Something goes wrong on the line for a while, then gets fixed (see the chart).
- **The process drifts.** The monthly failure rate fell from 23% in July to 3% in September. A sensor value that is normal one week can be a warning sign the next.



Daily failure rate. Bad days (orange) cluster together: failures come in bursts.

2. How our model works

The model combines three ideas. It does all the data cleaning itself, such as filling in missing sensor values, and it accepts files with the same columns as the training data.

1. **Read the sensors.** Four models each look at the 590 readings and vote. One reads the raw values. Two compare each wafer with the wafers made in the 3 days around it, which cancels out the slow drift. One is a different type of model (gradient boosting) that catches other patterns. We used random forests and LightGBM.
2. **Look at the neighbours in time.** A small model checks how many wafers made just before and after this one failed (within 2, 12 and 48 hours). It only uses the labels of the *training* wafers, and never a wafer's own result.
3. **Combine and decide.** The two opinions are combined (60% sensors, 40% time). A wafer is flagged when its failure risk is above the normal failure rate (6.6%). That cut-off is built into the saved model. With the usual 50% cut-off, the model would flag almost nothing and miss every failure.

Being open about the time feature. It looks at wafers made both *before and after* each wafer. That is fair for this test, because the test wafers come from the same period. It also matches real factory practice: when one wafer fails, its neighbours get re-checked. A factory that must decide the moment a wafer is made could only look *backwards*. That version still keeps about half of the improvement.

3. How we tested it, and the scores

We never scored a model on wafers it had trained on. We split the data into 5 parts many times over (train on 4, test on 1). We also ran 20 "practice leaderboards" the same size as the real hidden test. A change was kept only if it helped across all of these checks.

- **PR-AUC** measures how well the model ranks the bad wafers at the top of its list. Random guessing scores 0.066.
- **Balanced Accuracy** is the average of the share of bad wafers caught and the share of good wafers passed. A coin flip scores 0.50.

Model	PR-AUC	Balanced Accuracy
Our final model (cross-validation / practice leaderboards)	**0.262 / 0.290**	**0.714 / 0.703**
Sensors only, no time ideas	0.189 / 0.226	0.690 / 0.681
Published benchmark for this dataset (UCI)	n/a	0.665
Random guessing	0.066	0.500

The hidden test has only about 21 faulty wafers, so the real score can easily land a few points either way by luck (about ±0.06 Balanced Accuracy).

4. What this means for a factory

- **A smart second check.** Out of every 100 faulty wafers, the model catches about 77, while asking to re-check only 36% of all wafers. A flagged wafer is twice as likely to be faulty as a random one.
- **One dial to set.** If missing a bad wafer is very costly, lower the cut-off to catch more (up to 98%, with more checking). If checking is expensive, raise it.
- **Act on bursts.** When a wafer fails, check the wafers made around the same time first.
- **Where to look.** Sensors 16, 59, 477, 64 and 341 mattered most. They are good starting points for engineers. The sensor names are hidden in this dataset.

What didn't help: balancing the classes artificially, keeping only the "best" sensors, many other model types and settings, and other ways of using time. We kept only changes that held up in every check. **Limits:** with just 83 failures, the model should be re-checked on new production data.